Doc. no. P0001R1
Date: 2015-10-21
Project: Programming Language C++
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Revision History

Revision 0

Original version of the paper for the 2015 pre-Kona mailing.

Revision 1

Revisions following core wording review:

- Added compatibility-with-C clause.
- C++14 compatibility rationale explicitly mentions why the deprecated feature is removed, rather than why the keyword is preserved (it is obviously preserved if it is not removed!)

Remove Deprecated Use of the `register` Keyword

I. Table of Contents

II. Introduction

This paper is the first revision of N4340, which was approved to progress to Core from EWG at the April 2015 Lenexa meeting.

The `register` keyword was deprecated in the 2011 C++ standard, as its effect was already implicit in the language. It remains reserved for future use by the standard, and is time to remove its vestigial specification.

III. Motivation and Scope

One theme of C++17 appears to be cleaning up older features, such as trigraphs (N4086), `operator++(bool)` (Core issue 1653),

`auto_ptr` and the deprecated adaptable library function API (N4190). The `register` keyword has no normative function, yet occupies a place in the grammar that must be specified, so seems a good candidate for similar cleanup.

The `register` keyword was deprecated in the 2011 C++ standard in order to preserve our ability to repurpose a useful English keyword in future proposals, much as `auto` was repurposed in C++11. Its usage was deprecated, rather than removed, at the time as there was no conflicting immediate need for a proposed re-use at the time, unlike with `auto`. Similarly, the meaning of the `export` keyword was simply removed in C++11 due to implementation difficulty and a general lack of compilers implementing the feature. It was deemed appropriate to be more conservative with the widely implemented `register` keyword. However, users will have seen two standards published with the keyword deprecated, and with the general theme of clean-up occurring in C++17, should expect this advertised (potential) removal to finally occur.

One reason to consider retaining the keyword is compatibility with the C language. In particular, the `register` keyword can appear in a function parameter list. However, this usage has no meaning in C++, which may be problematic if calling into C code that ascribes a specific meaning to this hint.

However, C has further restrictions on `register` that do not translate to C++, that would impact only inline function definitions in shared headers. Code is ill-formed in C if it tries to take the address of a variable with the `register` *storage-class-specifier*. If an array is declared with the `register` *storage-class-specifier* in C, then it is undefined behavior to trigger array-to-pointer decay on that array.

Another comparison with C is the use of C99 keyword `restrict` in specifying interfaces. This seems much more widespread than the use of `register`, and has not proved to be a serious compatibility problem in the following 15 years.

Rather than try to resolve the differences with C, this paper prefers to remove the otherwise redundant feature from C++17.

IV. Impact On the Standard

The impact on the standard is minimal. The keyword is specified to be valid in a couple of places in the grammar, but explicitly has no meaning when used. It can safely be removed without impacting the integrity of the standard.

Looking beyond the standard: most of the early feedback to this paper has been that there appears, anecdotally, to be minimal use of the deprecated `register` keyword but considerably more use of the gcc extension based on this keyword. This suggests that future applications of the keyword may face practical constraints. Similarly, we should think very carefully before unreserving the keyword, even if we do not make immediate use of it again in the standard.

V. Design Decisions

Preferred Design

The purpose in removing the usage of `register` is to preserve the ability to use this valuable keyword for future proposals, such as modules. It is therefore retained as a reserved keyword, with a note on intent, similar to the handling of `export` in C++11. All other references to the `register` keyword are removed.

Alternative Design 1 (*not specified*)

If interface compatibility with C is important enough to retain that usage, then we should undeprecate the use of `register` when declaration function parameters, potentially restricted to parameters of functions declared with `extern "C"` linkage.

We might also consider support for `restrict` as a contextual keyword for functions declared with `extern "C"` linkage as part of the same design concern.

An updated paper along these lines would continue to advocate the removal of `register` as a *storage-class-specifier* for local variables though.

Alternative Design 2

The other suggestion received while authoring this paper is that it does not go far enough, and the `register` keyword should be released as an identifier to the user community.

`register` is certainly a flexible word offering significant use as both a verb (popular choice for function names, such as registering with a factory facility) and as a noun (for variables and class types).

If no significant proposal comes forward making use of `register` in the near future, it would be simple to amend 2.11 to no longer reserve the keyword. This paper recommends the more conservative approach for now though, of continuing to reserve a valuable term for standard use, as it would be difficult to reclaim once released.

EWG Recommendation

The EWG forwarded this proposal to Core, adopting the preferred diection above, with a poll in favor:

SF	F	N	A	SA
7	10	3	2	1

Design alternative 2 was polled and rejected:

SF	F	N	A	SA
0	0	3	12	5

The option to retain and undeprecate the `register` keyword was also polled:

SF	F	N	A	SA
1	1	1	12	7

VI. Technical Specification

Make the following edits (relative to n4296) with insertions and ~~removals~~ marked like so:

2.11 Keywords [lex.key]

1 The identifiers shown in Table 3 are reserved for use as keywords (that is, they are unconditionally treated as keywords in phase 7) except in an *attribute-token* (7.6.1) [*Note:* The `export` and `register` keywords are ~~is~~ unused but ~~is~~are reserved for future use. *â€" end note*]:

3.7.3 Automatic storage duration [basic.stc.auto]

1 Block-scope variables ~~explicitly declared `register` or~~ not explicitly declared `static` or `extern` have *automatic storage duration*. The storage for these entities lasts until the block in which they are created exits.

7.1.1 Storage class specifiers [dcl.stc]

1 The storage class specifiers are

```
storage-class-specifier:
    register
    static
    thread_local
    extern
    mutable
```

At most one *storage-class-specifier* shall appear in a given *decl-specifier-seq*, except that `thread_local` may appear with `static` or `extern`. If `thread_local` appears in any declaration of a variable it shall be present in all declarations of that entity. If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* of the declaration shall not be empty (except for an anonymous union declared in a named namespace or in the global namespace, which shall be declared `static` (9.5)). The *storage-class-specifier* applies to the name declared by each *init-declarator* in the list and not to any names declared by other specifiers. A *storage-class-specifier* other than `thread_local` shall not be specified in an explicit specialization (14.7.3) or an explicit instantiation (14.7.2) directive.

2 ~~The `register` specifier shall be applied only to names of variables declared in a block (6.3) or to function parameters (8.4). It specifies that the named variable has automatic storage duration (3.7.3).~~ [*Note:* A variable declared without a *storage-class-specifier* at block scope or declared as a function parameter has automatic storage duration by default (3.7.3). *â€" end note*]

~~3 A `register` specifier is a hint to the implementation that the variable so declared will be heavily used. [*Note:* The hint can be ignored and in most implementations it will be ignored if the address of the variable is taken. This use is deprecated (see D.2). *â€" end note*]~~

7.6.2 Alignment specifier [dcl.align]

1 An *alignment-specifier* may be applied to a variable or to a class data member, but it shall not be applied to a bit-field, a function parameter, or an *exception-declaration* (15.3)~~, or a variable declared with the `register` storage class specifier~~. An *alignment-specifier* may also be applied to the declaration or definition of a class (in an *elaborated-type-specifier* (7.1.6.3) or *class-head* (Clause 9), respectively) and to the declaration or definition of an enumeration (in an *opaque-enum-declaration* or *enum-head*, respectively (7.2)). An *alignment-specifier* with an ellipsis is a pack expansion (14.5.3).

8.3 Meaning of declarators [dcl.meaning]

2 A `static`, `thread_local`, `extern`, ~~`register`,~~ `mutable`, `friend`, `inline`, `virtual`, or `typedef` specifier applies directly to each *declarator-id* in an *init-declarator-list*; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its declarator.

9.2 Class members [class.mem]

5 A member shall not be declared with the `extern` ~~or register~~ *storage-class-specifier*. Within a class definition, a member shall not be declared with the `thread_local` *storage-class-specifier* unless also declared `static`.

A.6 Declarations [gram.dcl]

storage-class-specifier:
~~`register`~~
`static`
`thread_local`
`extern`
`mutable`

C.1.6 Clause 7: declarations [diff.dcl]

7.1.1

Change: In C++, `register` is not a storage class specifier.

Rationale: The storage class specifier had no effect in C++.

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Syntactic transformation.

How widely used: Common.

C.4.2 Clause 7: declarations [diff.cpp14.dcl.dcl]

7.1.1

Change: Remove `register` *storage-class-specifier*.

Rationale: Enable repurposing of deprecated keyword in future revisions of this International Standard.

Effect on original feature: A valid C++ 2014 declaration utilizing the `register` *storage-class-specifier* is ill-formed in this International Standard. The specifier can simply be removed to retain the original meaning.

~~D.2 `register` keyword [depr.register]~~

~~1 The use of the `register` keyword as a *storage-class-specifier* (7.1.1) is deprecated.~~

VII. Acknowledgements

Thanks for early reviews of this paper, while acknowledging that does not imply endorsing its contents, by Alan Bellingham, Bret Brown, John Fletcher, Al Grant, Kevlin Henney, Mike Miller, Nathan Myers, Roger Orr, Richard Smith, Ville Voutilainen, Jonathan Wakely, and Derek Woolverton.

VIII. References

- N4086 Removing trigraphs??!, Richard Smith
- N4168 Removing `auto_ptr`, Billy Baker
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- Core issue 809 Deprecation of the `register` keyword
- Core issue 1653 Removing deprecated increment of `bool`